



UNIVERSITÀ DI PISA

Data Mining Project Report

Bardazzi Carlo (603930)

Martini Virginia (667577)

Stira Alma (658699)

1. Introduction and Dataset Overview	3
2. Assessing data quality	3
2.1 Missing values	3
2.2 Duplicates	3
2.3 Errors and semantics inconsistencies	3
2.4 Further analysis	4
3. Correlations and irrelevant attributes	6
4. Outlier Detection and Dimensionality	8
4.1 Local Outlier Factor (LOF).....	8
4.2 ABOD (Angle Based Outlier Degree).....	9
4.3 LODA (Lightweight On-line Detector of Anomalies).....	9
4.4 Isolation Forest	10
4.5 Conclusion	11
5. Imbalanced Learning.....	11
6. Advanced Classification	12
6.1 Regressione Logistica.....	12
6.2 Support Vector Machines (SVM).....	13
6.3 Reti Neurali	13
6.4 Metodi di Ensemble.....	13
6.4.1 Random Forest.....	14
6.4.2 Gradient Boosting.....	14
7. Advanced Regression.....	15
7.1 Logistic Regression	15
7.2 Gradient Boosting.....	17
7.3 Comparison Metrics	18
8. Time Series Analysis	18
8.1 Data Understanding and Preparation.....	18
8.2 Clustering	19
8.2.1 Partitional Clustering.....	19
8.2.2 Hierarchical clustering.....	20
8.2.3 Features-based clustering	21
8.3 Classification	22
8.3.1 Pre-processing	23
8.3.2 Shapelets.....	23
8.3.3 Comparison between Shapelets and Motifs	25
8.3.4 K-NN	25

8.3.5	Random Forest.....	26
8.3.6	Gradient Boosting.....	26
8.3.7	ROCKET	27
8.3.8	Conclusioni.....	27
8.4	Motifs and Discords	27
9.	Explainability	28
9.1	Global Methods	28
9.1.1	Trepan (Tree-based Rule Extraction)	29
9.2	Local Methods	29
9.2.1	SHAP Values (SHapley Additive exPlanations).....	30
9.2.2	LIME (Local Interpretable Model-agnostic Explanations)	30
9.3	Conclusion.....	31

1. Introduction and Dataset Overview

The "tracks.csv" dataset is a rich and detailed collection of information on 109,547 musical tracks from Spotify. The dataset contains 34 columns, each representing a specific attribute of a musical track. These attributes provide a comprehensive overview of the musical characteristics and popularity of the tracks.

Musical tracks can be analyzed through a variety of features to better understand their properties and popularity. In this project, we aim to explore and analyze a dataset of musical tracks to investigate various musical attributes and their impact on track popularity and genre classification. The dataset includes features such as danceability, energy, key, loudness, among others, offering a complete view of each track's musical profile.

Firstly, in Section 2, various data preparation and understanding operations are performed, including attribute transformation and elimination, followed by data splitting useful for subsequent experiments. Subsequently, in Section 4, an outlier detection phase is conducted, utilizing dimensionality reduction techniques as well. Then, in Section 5, an imbalanced class scenario is tested, where the dataset presents a natural imbalance in the classes. It was decided to test two algorithms for oversampling and one for undersampling. Following this, a classification task is performed to showcase and compare the results. In Section 6, various multi-class classification algorithms and multiple feature regression are executed. For the classification task, it was decided to classify the tracks into different genres based on their attributes.

2. Assessing data quality

2.1 Missing values

Within our dataset, a thorough analysis was conducted to identify any missing values. This verification confirmed the complete absence of missing values in the data at our disposal.

2.2 Duplicates

During the analysis of our dataset, we identified a total of 33,674 elements that exhibited significant similarity in their main attributes. The primary differences observed included variations in popularity, differing by only one point, discrepancies in artist names, sometimes spelled slightly differently, and variations in music genre. To address differences in popularity, we opted to select the lower value among those available. For discrepancies in artist names, we standardized the naming by considering the most frequent or correct form. Regarding music genres, we categorized a portion of the data and treated the remaining differences as distinct songs, yet we merged similar items under a single genre, listing multiple types separated by commas. These steps enabled us to consolidate and improve the coherence of the data, thereby enhancing the overall quality of our analysis and results obtained.

2.3 Errors and semantics inconsistencies

During the analysis of the variable `time_signature`, two noteworthy observations emerged. Firstly, a discontinuity in the values was noticed, with the absence of the value "2". This anomaly suggested the possibility of a data error, implying an accidental jump in values. To address this inconsistency, one unit was subtracted from all instances where the values were equal to 3, 4, and 5. Additionally, concerning the variable `time_signature`, the inconsistency of having null values (0) was detected, which lacks logical meaning as a time signature cannot be 0/4. Therefore, to maintain data coherence and validity, it was decided to replace the `time_signature` values of 0 with the mode of other songs within the same genre. These corrections were

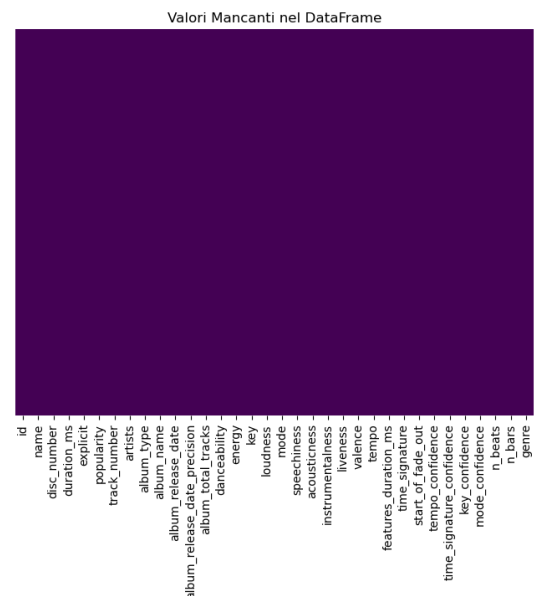


Figure 1: Missing values

implemented to ensure clarity and reliability in the data, avoiding ambiguities or misinterpretations in subsequent phases of the analysis.

Regarding the columns key and mode, it was decided to merge them into a single column. This choice was motivated by both logical and semantic reasons. Merging the two columns allows for a more consistent and meaningful representation of the tonal characteristics of the songs. In fact, the combination of key (which indicates the musical key) and mode (which distinguishes between major and minor modes) provides a complete and unique description of the harmonic context of each musical piece. By uniting these two columns, a more accurate representation of the musical properties is achieved, facilitating data interpretation and analysis. Additionally, this unification simplifies the process of data processing and manipulation, reducing the complexity of subsequent analyses. This decision was made to improve the quality and consistency of the information within the dataset, ensuring greater effectiveness of analytical and predictive analyses.

During the in-depth analysis of the features in our dataset, it was observed that the temporal variable consisted of nothing more than the product of the track duration, expressed in minutes, and the number of beats (n_beats), so we decided to delete the attribute.

2.4 Further analysis

Moreover, during these analyses, since there were a total of 114 genres and many of them were similar, it was decided to merge them into macro-families as follows:

1. Soft Electronic: edm, idm, house, trance, deep-house, electro, electronic, chicago-house, minimal-techno, techno, drum-and-bass, progressive-house, breakbeat, synth-pop, garage, dub
2. Dance: dance, disco, dancehall, j-dance, club, party
3. Pop: pop, indie-pop, k-pop, j-pop, pop-film, power-pop, pop-rock, cantopop, mandopop, j-idol, afrobeat, hip-hop, trip-hop
4. Hard Electronic: hardcore, detroit-techno, dubstep, breakbeat, hardstyle, industrial
5. Rock: synth-pop, pop-rock, rock, punk-rock, rock-n-roll, alt-rock, j-rock, psych-rock, new-age, grunge
6. Metal: heavy-metal, metal, hard-rock, black-metal, metal-core, death-metal, grindcore, groove
7. Other: industrial, alternative, indian, iranian, anime, swedish, turkish, world-music, french, german, malay, british, show-tunes, comedy, happy
8. Punk: emo, goth, punk, punk-rock
9. Latin: spanish, mpb, samba, salsa, pagode, forro, latin, latino, reggae, reggaeton, tango, sertanejo, dub, brazil
10. Romance: sertanejo, romance, guitar, acoustic, songwriter, singer-songwriter, piano, sad, indie
11. Classical: classical, opera, piano
12. Chill: chill, study, sleep, ambient
13. R&B: r-n-b, jazz, soul, gospel, blues, ska
14. Folk: folk, blues, country, bluegrass, honky-tonk
15. Children: children, kids, Disney

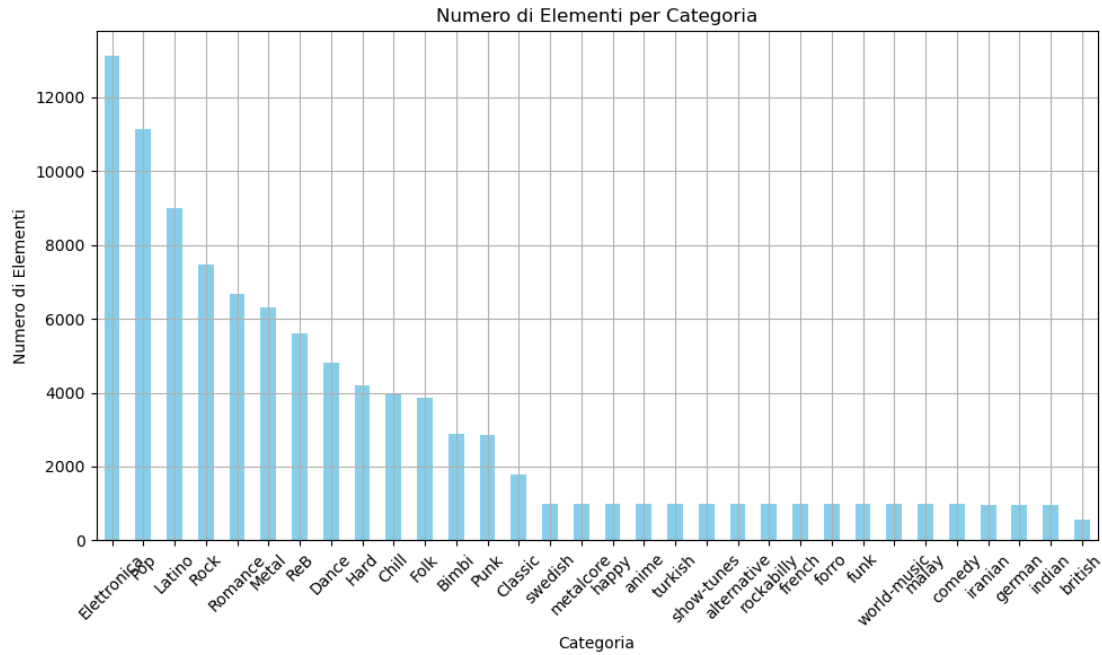


Table 3: Records per Genre

3. Correlations and irrelevant attributes

Correlation analysis was conducted to identify the relationships between the various characteristics of the musical tracks present in the "tracks.csv" dataset. Using Spearman's method, correlations between numerical variables were calculated. Below, we describe the main findings from the analysis:

1. Energy and Loudness (0.75):

- Positive Correlation: A strong positive correlation was observed between energy and loudness. This indicates that tracks with high energy levels also tend to be louder. Therefore, musical tracks with high energy, such as those in the rock and dance genres, often have higher volumes.

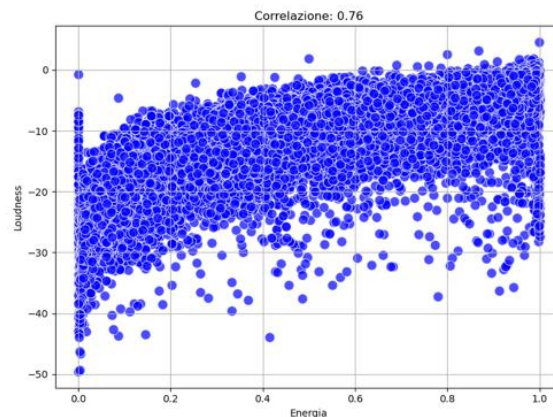


Table 4: Energy & Loudness Correlation

2. Danceability and Valence (0.46):

- Moderate Positive Correlation: There is a moderate positive correlation between danceability and valence. This suggests that tracks that are easy to dance to tend to have a more positive or cheerful tone. Genres like pop and dance music, which are generally danceable, often convey positive feelings.

3. `Duration_ms` and `N_Beats` (0.80):
 - Positive Correlation: The track duration is positively correlated with the number of beats (`n_beats`). Longer tracks naturally contain more beats due to their extended duration.
4. `Features_duration_ms` and `Start_of_fade_out` (0.99)
 - Strong Positive Correlation: This indicates that, generally, the longer the track, the later the fade-out starts. The relationship is almost linear, suggesting a direct connection between the track's length and the point at which it begins to fade out.

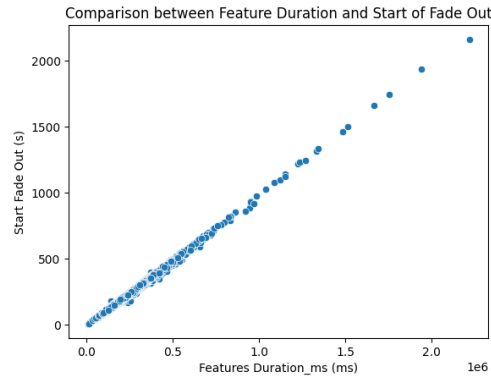


Table 5: `Features_duration_ms` & `Start_of_fade_out` Correlation

5. Acousticness and Energy (-0.71):
 - Strong Negative Correlation: A strong negative correlation was detected between acousticness and energy. Acoustic tracks tend to have lower energy levels, which is consistent with the quieter and less intense nature of acoustic music.

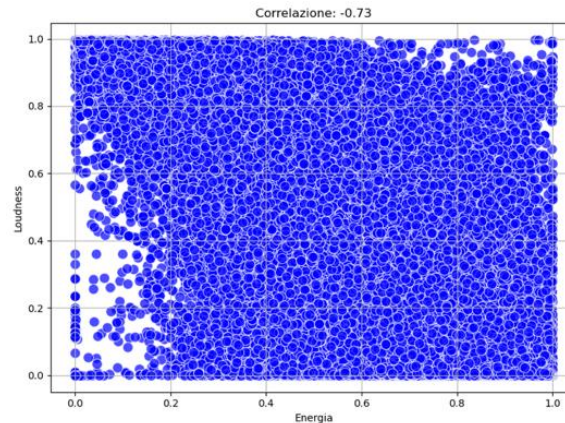


Table 6: Acousticness & Energy Correlation

The analysis of correlations between the musical characteristics of the tracks provides a detailed understanding of the internal dynamics of the dataset. These findings can be used to enhance music recommendations, create personalized playlists, and develop more accurate predictive models for the popularity and classification of musical genres.

4. Outlier Detection and Dimensionality

In this phase, we searched for outliers, which are values that can negatively impact model results. Therefore, it is essential to identify and manage them correctly. Specifically, we searched for the top 1% of the most extreme points, thus identifying the most significant anomalies, filtering out noise, and focusing on the points that could have the greatest impact on the system or model. We used this approach to reduce computational and memory load, improving the efficiency of the analyses, and because it is more likely to identify real anomalies, reducing false positives and enhancing the accuracy of subsequent analyses.

During our analysis, we did not consider the confidence attributes (tempo_confidence, time_signature_confidence, key_confidence, mode_confidence, popularity_confidence) because given the qualitative nature of this attribute, it was determined that values which might otherwise be considered outliers should not be excessively categorized as such. This decision was motivated by the intention to preserve vital information regarding the quality of the measurement of a track's popularity while maintaining an informative and contextually precise approach.

4.1 Local Outlier Factor (LOF)

Initially, to detect outliers, we used the LOF model, which is based on the local density of a point relative to its neighbors, classifying as outliers those points that have a significantly lower density compared to others. Once the outliers were detected, totaling 4,479 points, we applied PCA to reduce the dimensionality of the data and facilitate their visualization in a two-dimensional space. This resulted in a graph where the blue points represent normal data and the red points represent outliers. It is observed that the outliers tend to deviate significantly from the central mass of the data, confirming their anomalous nature.

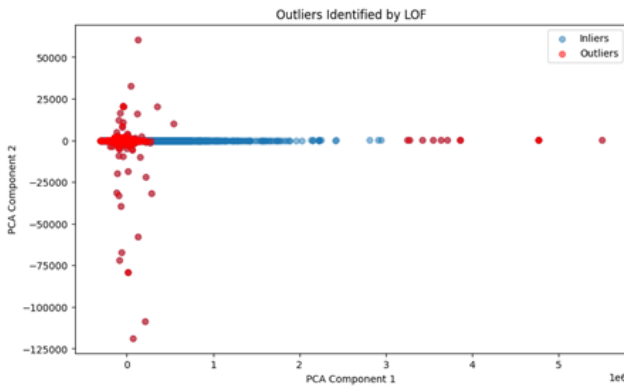


Figure 2: Outliers with LOF

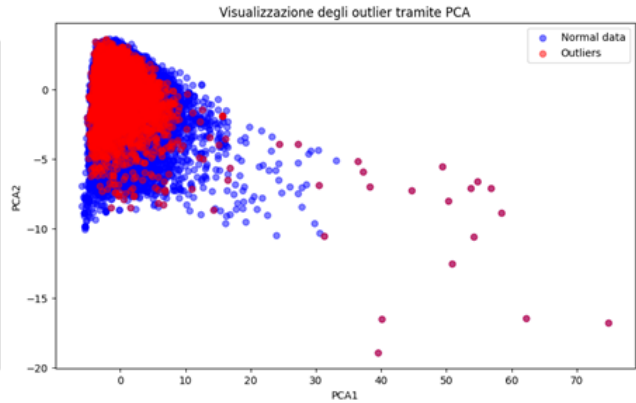


Figure 3: Outliers with PCA

The following charts illustrate the distribution of outliers across different categories and show how these anomalous points are distributed within the dataset.

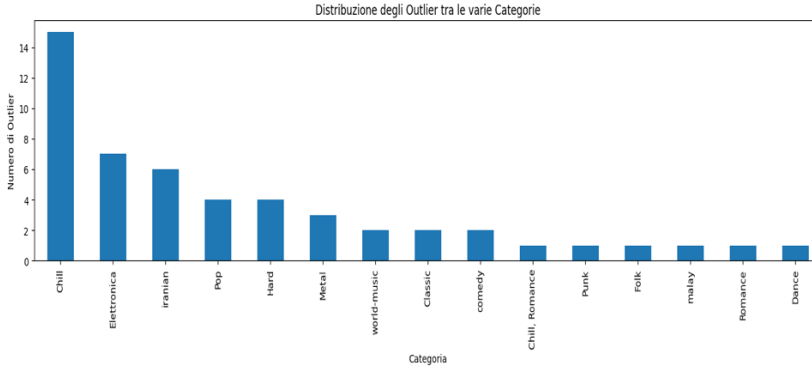


Table 7: Outliers per Genre

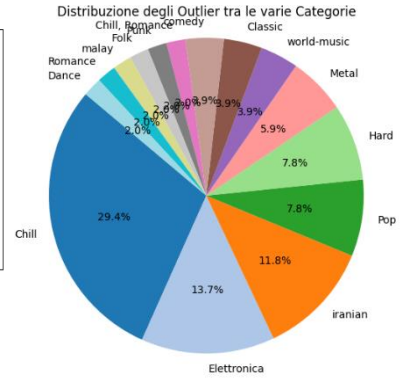


Table 8: Outliers per Genre

Here it is evident that certain categories, such as "Chill" and "Electronic," exhibit a significantly higher number of outliers compared to others. This might indicate that in these categories, there are more frequent occurrences of anomalous behaviors or data.

4.2 ABOD (Angle Based Outlier Degree)

The second algorithm used is ABOD, which is specifically designed for high-dimensional datasets. This method leverages angles to calculate the outlier score, as angles are more stable compared to distances in multidimensional spaces. Points considered as outliers are those around which the rest of the data points are positioned in similar directions. In other words, a point is considered normal if the variance of the angles is large, indicating that there are points distributed in every direction. Thus, outliers tend to be located at the edges of distributions. The model measures the variance of the spectrum of angles formed by the point in question with respect to all other points.

For this approach, the data was initially standardized using StandardScaler. PCA was then used to reduce the dimensionality of the data and facilitate the visualization of outliers in a two-dimensional space.

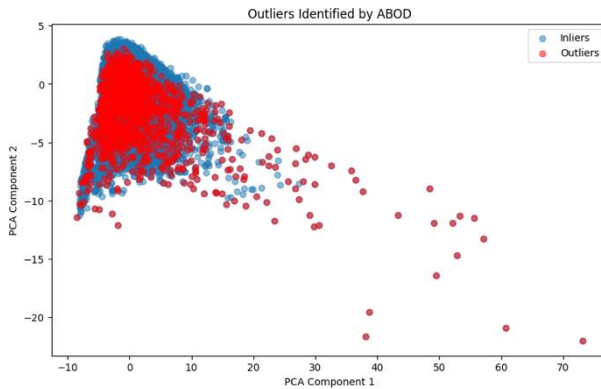


Figure 4: Outliers by ABOD

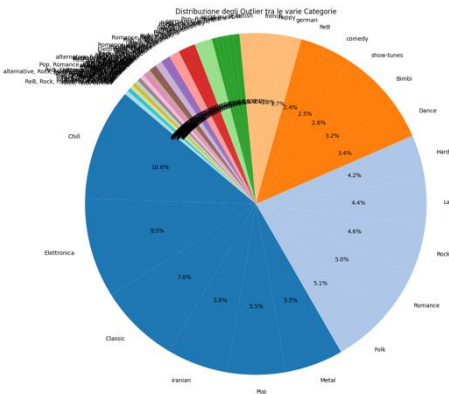


Figure 5: Outliers per Genre by ABOD

While LOF identified a significant number of outliers in the "Chill," "Electronic," and "Iranian" categories, ABOD highlights a higher number in the "Electronic," "Classic," and "Rock" categories.

4.3 LODA (Lightweight On-line Detector of Anomalies)

LODA is also an algorithm designed to detect anomalies in large datasets. It relies on the use of random projections of the data into lower-dimensional spaces, then combines the results of these projections to

calculate an anomaly score. In this case as well, we used StandardScaler and subsequently PCA for visualization.

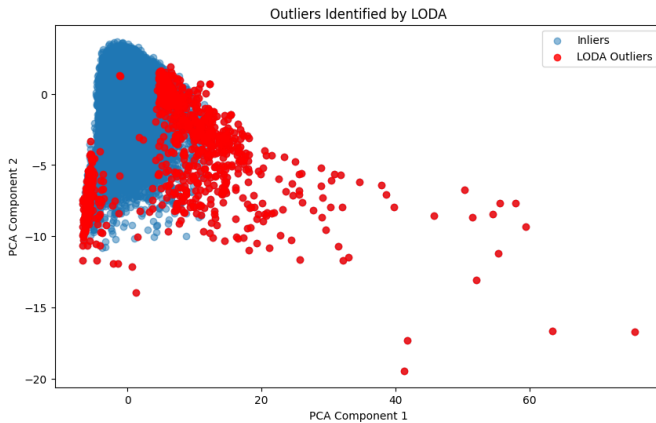


Figure 6: Outliers by LODA

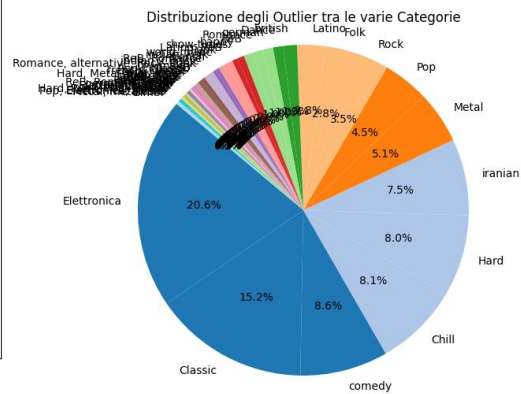


Figure 7: Outlier per Genre LODA

With this analysis, the categories with a high number of outliers are: "Electronic" and "Classic".

4.4 Isolation Forest

Isolation Forest is the last algorithm used for detecting outliers in our dataset. This algorithm belongs to the family of model-based approaches, which means it builds a model to classify the data and can also be used to evaluate new subsequent records. The basic idea of Isolation Forest is to construct a forest of decision trees. During the construction of the trees, dimensions and values to split the dataset are chosen randomly. This process continues until a minimum number of records in the leaf nodes or a certain tree depth is reached. Points that are frequently isolated in leaf nodes at higher tree levels are considered outliers. For each point in the dataset, a score is calculated based on all the trees built. A score closer to 1 indicates a higher probability that the point is an outlier, as it was isolated more quickly compared to other points.

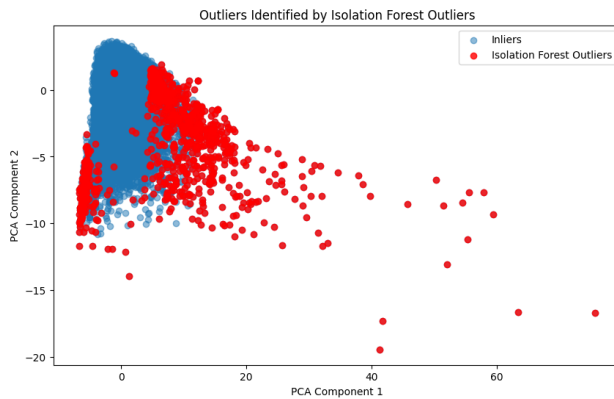


Figure 8: Outliers by Isolation Forest

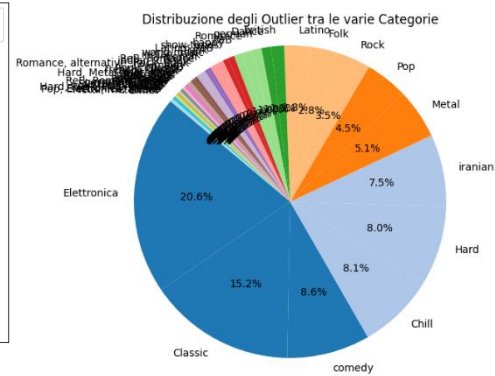


Figure 9: Outliers per Genre with Isolation Forest

This analysis further confirmed that the categories "Electronic," "Classic," and "Comedy" exhibit a significant number of anomalies.

4.5 Conclusion

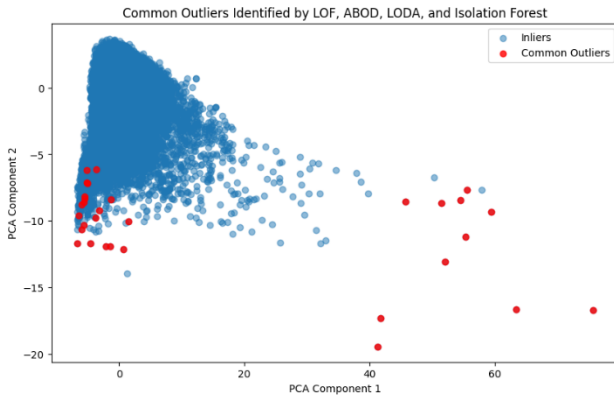


Figure 10: Common Outliers

To compare the results obtained from the various outlier detection algorithms, we identified the common outliers detected by the methods Local Outlier Factor (LOF), Angle-Based Outlier Detection (ABOD), Lightweight Online Detector of Anomalies (LODA), and Isolation Forest. Our initial analysis revealed that combining all the outliers detected by each of these methods resulted in an excessively high number of outliers. Consequently, we decided to adopt a more restrictive criterion, considering only the outliers detected by at least three of the mentioned algorithms.

Once the common outliers were identified according to this criterion, we experimented with different strategies to manage these anomalous values. The strategies tested included removing the outliers, replacing them with the mean value, the median value, or the most frequent value (mode). To determine the best strategy to adopt, we evaluated not only the number of outliers detected in the resulting new dataset but also how these strategies affected the data distributions.

From the analysis of the results, it emerged that replacing the outliers with the mean value represented the optimal solution. This methodology not only significantly reduced the number of outliers in the final dataset but also maintained data distributions that were more homogeneous and consistent with the original ones.

Therefore, we decided to adopt replacement with the mean value as the primary action plan for managing outliers in our data analysis process.

This methodological choice allowed us to preserve the integrity and quality of the data, facilitating more accurate and reliable analyses. In summary, replacing outliers with the mean value proved to be the most effective strategy for managing outliers, ensuring a reduction in their negative effects and improving the consistency of data distributions in the final dataset. Given the performance obtained from combining the various algorithms, we believe that identifying common outliers across multiple methods can increase the robustness of the analysis and provide a more solid foundation for making informed decisions and optimizing machine learning models.

5. Imbalanced Learning

In this study on imbalanced learning, the issue of an imbalanced dataset was addressed using various balancing techniques, focusing primarily on undersampling and oversampling with the Decision Tree and K-Nearest Neighbors (KNN) models.

In the undersampling section, two main approaches were explored: RandomUnderSampler (RUS) and ClusterCentroids (CC). With both techniques, both Decision Tree and KNN generally showed poor performance. The RUS slightly improved recall but with still low precision, while the CC maintained a more faithful data distribution but with overall unsatisfactory results.

Undersampling Techniques	KNN		Decision Tree	
RandomUnderSampler	Precision:	0.06	Precision:	0.20
	Recall:	0.07	Recall:	0.27
	F1-Score:	0.06	F1-Score:	0.20

ClusterCentroids	Precision:	0.03	Precision:	0.12
	Recall:	0.04	Recall:	0.14
	F1-Score:	0.03	F1-Score:	0.09

Moving on to oversampling, techniques such as SMOTE and RandomOverSampler were evaluated. SMOTE produced significant improvements for the Decision Tree, increasing precision, recall, and overall accuracy. However, KNN benefited less from the oversampling techniques, showing only slight improvements.

Overrsampling Techniques	KNN		Decision Tree	
RandomOverSampler	Precision:	0.08	Precision:	0.22
	Recall:	0.10	Recall:	0.25
	F1-Score:	0.08	F1-Score:	0.23
SMOTE	Precision:	0.09	Precision:	0.23
	Recall:	0.11	Recall:	0.26
	F1-Score:	0.09	F1-Score:	0.24

ADASYN was explored but failed to generate effective synthetic samples due to the extremely imbalanced distribution of classes in the music dataset under consideration.

The general conclusions highlighted that class balancing is crucial for improving the performance of machine learning models, especially in contexts with highly imbalanced datasets. SMOTE emerged as the most effective technique for this specific dataset, significantly enhancing the performance of the models, especially for the Decision Tree. Ultimately, the analysis emphasized the importance of carefully balancing datasets to avoid significant information loss (as in the case of undersampling) and to improve the predictive capability of machine learning models.

6. Advanced Classification

For this part of the analysis, we will address the previously defined classification task using various advanced classification methods: Logistic Regression, Support Vector Machines (SVM), Neural Networks, Ensemble Methods, and Gradient Boosting Machines. For each method, hyperparameter tuning will be performed and the performance will be evaluated using appropriate evaluation metrics.

6.1 Regressione Logistica

We applied logistic regression to our music category classification dataset. After splitting the dataset into training and test sets, we used the SMOTE method to balance the classes in the training set. Subsequently, we performed a grid search to optimize the hyperparameters of the logistic regression model, finding the best parameters: C=10, penalty='l2', and solver='liblinear'.

Model Performance:

- Accuracy: 0.20
- Average Precision (macro): 0.21
- Average Recall (macro): 0.32
- Average F1-Score (macro): 0.20
- ROC AUC Score: 84%

Despite the good ROC AUC Score, which indicates a good discrimination ability, the low accuracy and the values of precision, recall, and F1-score suggest that the logistic regression model is not sufficiently effective in correctly classifying all music categories. This could be due to the high complexity of the dataset and the imbalanced nature of the classes.

6.2 Support Vector Machines (SVM)

In this study, an SVM (Support Vector Machine) model was applied to the music category classification dataset. After performing a grid search to optimize the hyperparameters, the best parameters found were $C=1$, $\gamma='scale'$, and $kernel='linear'$.

To optimize computation time, the following modifications were made:

- Reduction of Parameter Grid: The number of parameters for kernel and gamma was reduced to speed up the search process.
- Reduction of Cross-Validation Folds: The cv value was changed from 5 to 3 to expedite cross-validation.
- Parallelization: The $n_jobs=-1$ parameter was added to GridSearchCV to utilize all available CPU cores.

Model Performance:

- Accuracy: 80%
- Average Precision: 78%
- Average Recall: 81%
- Average F1-Score: 79%

The SVM model proved to be very effective in classifying most of the music categories in the dataset. However, some classes require further improvement. It may be useful to explore advanced data preprocessing or class balancing techniques or to try other more complex model architectures to further enhance performance.

The overall results suggest that the SVM with a linear kernel is a valid choice for this type of classification, offering a good balance between precision and recall for most classes.

6.3 Reti Neurali

Neural networks offer great flexibility and are particularly powerful for complex problems. Although they require more time for training and a more complex hyperparameter tuning phase, they have shown good performance in terms of F1-score. The best parameters found were: $learning_rate_init=0.001$, $hidden_layer_sizes=(50,)$, and $\alpha=0.01$.

Model Performance:

- Accuracy: 0.29
- Precision, Recall, and F1-Score: Metrics vary significantly between categories.
- ROC AUC Score: An average AUC of 0.89 suggests a good ability to distinguish between classes.

The neural network model has shown good potential in classifying music categories, with promising results across various evaluation metrics.

6.4 Metodi di Ensemble

Ensemble methods such as Random Forest and Gradient Boosting combine multiple weak learners to improve performance.

Both ensemble methods demonstrated good performance and robustness. In particular, Gradient Boosting proved to be very effective in improving classification performance. These methods combine the predictions of many weak learners to produce a robust and accurate model.

6.4.1 Random Forest

The Random Forest model, optimized with the parameters found through GridSearchCV {'n_estimators': 200, 'min_samples_split': 5, 'min_samples_leaf': 1, 'max_depth': 30, 'bootstrap': False}, showed excellent performance in classifying music categories. Here is a detailed analysis of the performance metrics:

Performance Metrics:

- Accuracy: 0.85
- Precision: 0.83 (macro)
- Recall: 0.90 (macro)
- F1-Score: 0.85 (macro)
- ROC AUC Score: 0.99

Therefore, the model proved to be highly effective for this music category classification task. The combination of high precision, recall, and AUC suggests that the model is well-balanced and suitable for practical applications. The combination of robustness, feature handling capability, overfitting mitigation, and the effective use of balancing techniques and feature engineering made the Random Forest model particularly effective for music category classification.

6.4.2 Gradient Boosting

The Gradient Boosting model showed remarkable performance on the test data. The best parameters found were: {'subsample': 0.8, 'n_estimators': 200, 'min_samples_split': 10, 'max_depth': 5, 'learning_rate': 0.1}.

Gradient Boosting Model Performance:

- Precision: 0.84
- Recall: 0.87
- F1-Score: 0.85
- ROC AUC Score: 1.00

The Gradient Boosting model proved to be extremely effective for the task of music category classification, achieving excellent performance across various evaluation metrics. With an overall accuracy of 86%, a precision of 84%, a recall of 87%, and an F1-Score of 85%, the model demonstrated robust and reliable classification capabilities. Additionally, the perfect ROC AUC Score of 1.00 indicates that the model is highly proficient in distinguishing between the various classes.

In summary, Gradient Boosting has proven to be a powerful and versatile method for multiclass classification, capable of handling complex datasets and providing accurate and consistent results.

6.1 Conclusion

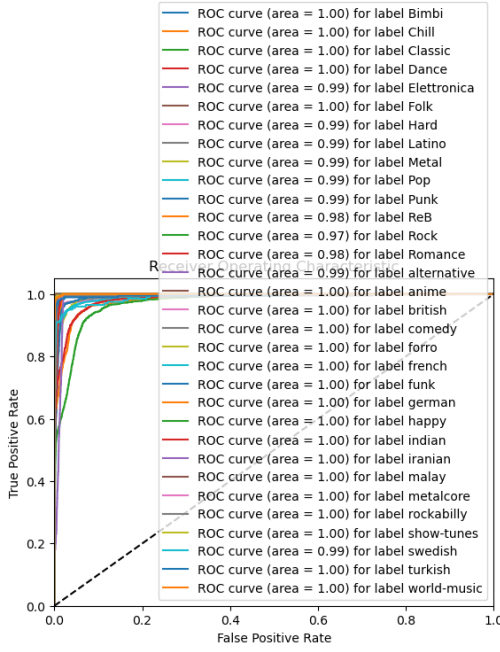


Figure 11: ROC for Gradient Boosting

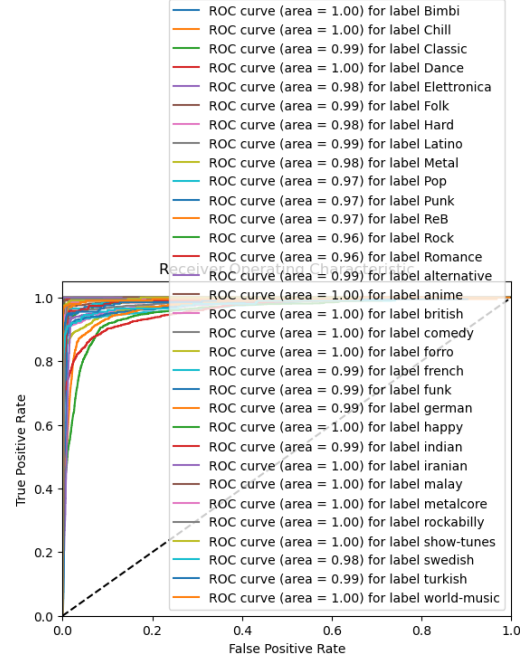


Figure 12: ROC for Random Forest

The ensemble models, particularly Random Forest and Gradient Boosting, showed the best performance for our task of music category classification. Their ability to combine the predictions of multiple weak learners resulted in robust and accurate outcomes. Neural networks, although requiring more computational resources and training time, demonstrated good potential and could benefit from further improvements and tuning. Logistic regression, while simpler to implement, showed performance limitations due to the complexity of the dataset. Finally, SVM proved to be a valid model with good performance, particularly effective for some classes.

7. Advanced Regression

In this section, we will discuss and analyze the regression tasks. The objective of these tasks was to determine if a track is popular. First, we binarized our variable by setting it to 1 if the popularity had a value greater than 70, and 0 otherwise. We used logistic regression and gradient boosting as our approaches.

For the analysis, we considered all attributes that express a quality of the track, as well as genre, key/mode, and time signature, on which we performed one-hot encoding.

7.1 Logistic Regression

After splitting the dataset into training and test sets, we performed a grid search to optimize the model's hyperparameters, finding $C=10$, `solver='lbfgs'`, and `class_weight='balanced'`.

We chose $C=10$ to allow the model to better fit the training data, as we did not expect much noise in the dataset. The “lbfgs” solver was chosen due to the large number of variables considered for the initial analysis and the large size of the dataset. Additionally, we set `class_weight` to “balanced” to address the significant disparity in the number of elements in the popularity variable.

	precision	recall	f1-score	support
0	0.98	0.60	0.74	29155
1	0.07	0.68	0.12	1232
accuracy			0.60	30387
macro avg	0.52	0.64	0.43	30387
weighted avg	0.94	0.60	0.72	30387

Table 9: Logistic Regression Analytic Metrics – all feature

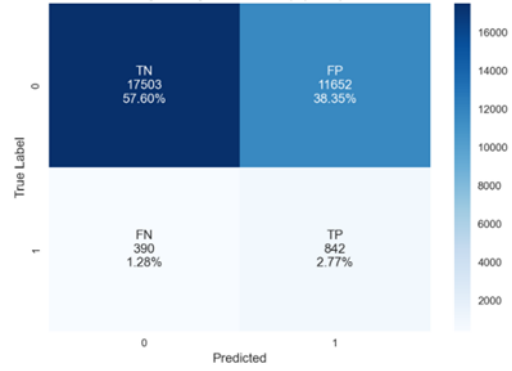


Table 10: Confusion Matrix Logistic Regression – all feature

After an initial analysis, we examined the regression coefficients to understand which features better defined popularity.

Nomi delle features con coefficienti positivi: 'explicit', 'danceability', 'loudness', 'valence', 'categoria_Chill', 'categoria_Dance', 'categoria_Elettronica', 'categoria_Folk', 'categoria_Latino', 'categoria_Metal', 'categoria_Pop', 'categoria_Punk', 'categoria_ReB', 'categoria_Rock', 'categoria_Romance', 'categoria_alternative', 'categoria_british', 'categoria_french', 'categoria_funk', 'categoria_german', 'categoria_indian', 'categoria_rockabilly', 'categoria_show-tunes', 'categoria_swedish', 'time_signature_1', 'time_signature_3', 'time_signature_4', 'chord_type_n_1', 'chord_type_n_2', 'chord_type_n_5', 'chord_type_n_7', 'chord_type_n_8', 'chord_type_n_9', 'chord_type_n_12', 'chord_type_n_16', 'chord_type_n_17', 'chord_type_n_18', 'chord_type_n_20', 'chord_type_n_21', 'chord_type_n_22'

Figure 13: Features with positive coefficients

After various trials, we observed that the main features that help identify popularity are: explicit, loudness, time signature, and key/mode. Additionally, it can be noted that the most popular genres are: Electronic, Folk, Metal, Pop, R&B, Rock, Romance, and Alternative.

	precision	recall	f1-score	support
0	0.99	0.64	0.77	29155
1	0.08	0.79	0.15	1232
accuracy			0.64	30387
macro avg	0.54	0.71	0.46	30387
weighted avg	0.95	0.64	0.75	30387

Figure 14: Logistic Regression with Feature Selection Analytic Metrics

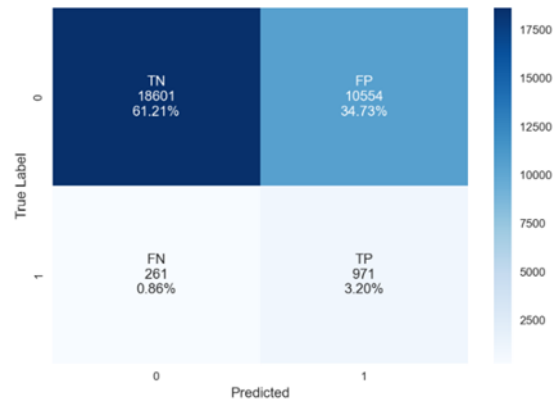


Figure 15: Confusion Matrix

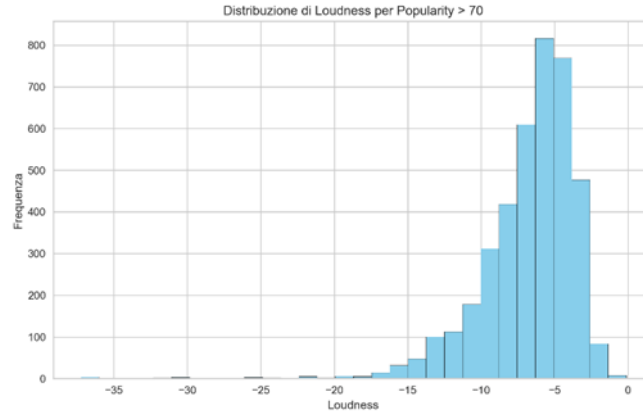


Table 11: Loudness Distribution per Popularity

7.2 Gradient Boosting

After splitting the dataset into training and test sets, we performed a grid search to optimize the model's hyperparameters. The best parameters found were: {'n_estimators': 200, 'min_samples_split': 10, 'max_depth': 7, 'learning_rate': 0.1}.

For selecting the model's parameters, we focused mainly on obtaining a slightly slower model that would go into more detail to achieve better results, without worrying too much about noise due to the previous modifications to the dataset that should have reduced it considerably.

In this case as well, we initially used the same features as for Logistic Regression and then narrowed them down based on the feature importance of the model. From this analysis, it emerged that, unlike Logistic Regression, the attributes that most help understand popularity are those that express a characteristic of the song, including: Duration_ms, Energy, Loudness, Speechiness, Acousticness, Instrumentalness, Liveness, and Valence

	precision	recall	f1-score	support
0	0.96	1.00	0.98	29155
1	0.00	0.00	0.00	1232
accuracy			0.96	30387
macro avg	0.48	0.50	0.49	30387
weighted avg	0.92	0.96	0.94	30387

Figure 16: Gradient Boosting Feature Selection
Analytic Metrics

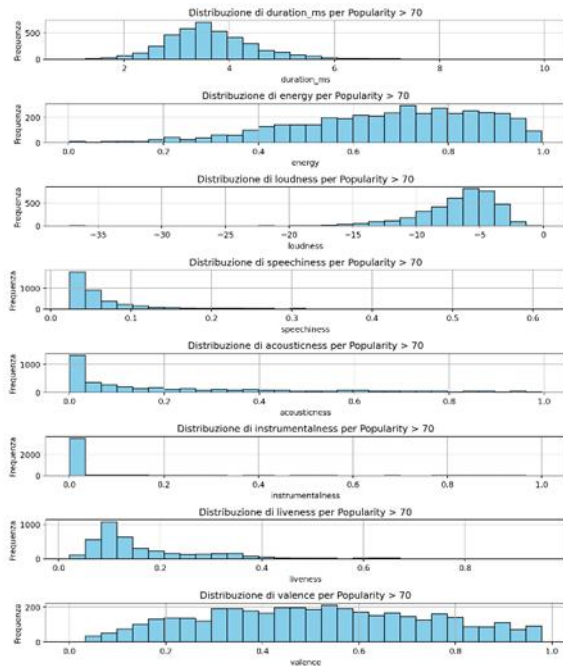


Figure 17: Feature Distribution

7.3 Comparison Metrics

Mettriche	Logistic Regression	Gradient Boosting
Mean Absolute Error (MAE)	0.35590877677954386	0.040675288774805016
Mean Squared Error (MSE)	0.35590877677954386	0.040675288774805016
Root Mean Squared Error (RMSE)	0.5965809054768212	0.20168115622141056
R-squared (R^2)	-8.14935747026083	-0.04564085374409488

The first set of metrics indicates that the model has very poor performance. The MAE, MSE, and RMSE are very high, suggesting that the model predicts significantly incorrectly compared to the observed values. Additionally, the very negative R^2 value confirms that the model is ineffective at making predictions better than a simple mean estimate.

In the second set of metrics, the model shows very low MAE, MSE, and RMSE, indicating that the predictions are very close to the observed values. However, the negative R^2 value (-0.0456) suggests that the model fails to explain the variation in the data better than a simple model predicting the mean of the data. It may be necessary to examine the model more closely to improve its predictive ability.

8. Time Series Analysis

8.1 Data Understanding and Preparation

Within the time series, 20 genres are identified, which differs from the tabular dataset. Since we redefined the genres in the dataset, we also examined any resulting changes here. After converting the data to standardize the representation of genres, we observed a reduction from 20 to 13. Although the new division did not show significant imbalances among the different genres, we decided to maintain the original partition. This decision was made to avoid imbalances in the number of samples for each genre. However, we kept the new division in mind during the analysis, considering the possibility of discovering significant insights that might emerge.

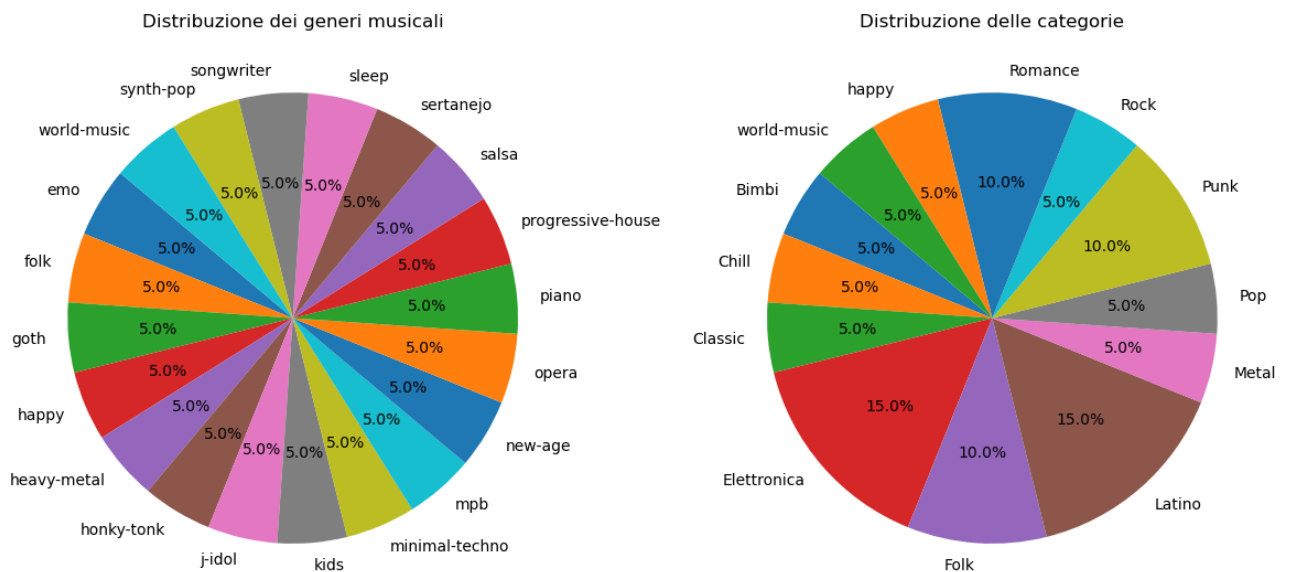


Figure 18: Distribution per Genre

The process began with the loading and normalization of the time series. Using techniques such as the MinMaxScaler, all series were scaled to have values between 0 and 1, thus ensuring a uniform scale. This step was crucial to ensure that all features of the time series had the same weight during the clustering process.

8.2 Clustering

In this section of the project, we used three distinct methodologies for data analysis: Partitional, Hierarchical, and Feature-based. Within these methodologies, we applied two different distance measures: Euclidean distance and Dynamic Time Warping (DTW) distance.

8.2.1 Partitional Clustering

Two clustering methods were applied: KMeans with Euclidean distance and KMeans with DTW (Dynamic Time Warping) distance. For both methods, evaluation metrics were calculated without any initial preprocessing. The results were as follows:

- Silhouette Score Euclidean: 0.2224
- Davies-Bouldin Index Euclidean: 1.4420
- Silhouette Score DTW: 0.2194
- Davies-Bouldin Index DTW: 1.4418

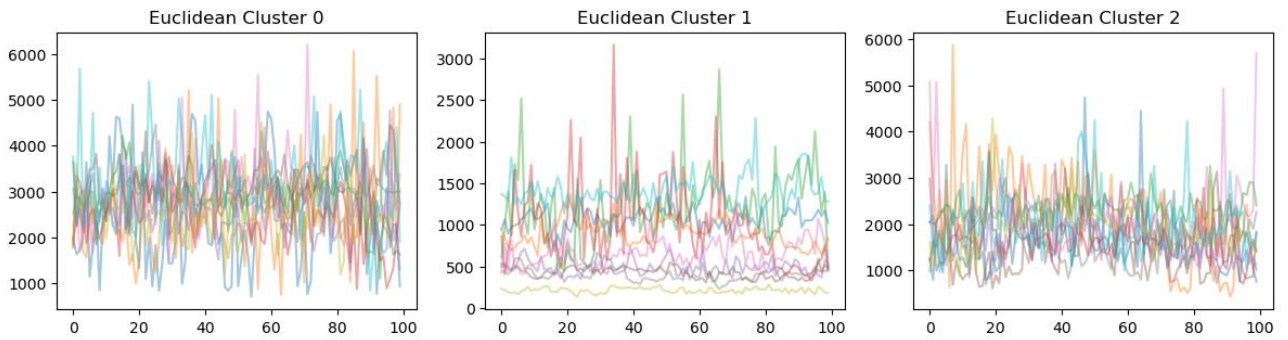


Table 13: Partitional Clustering with Euclidean Distance

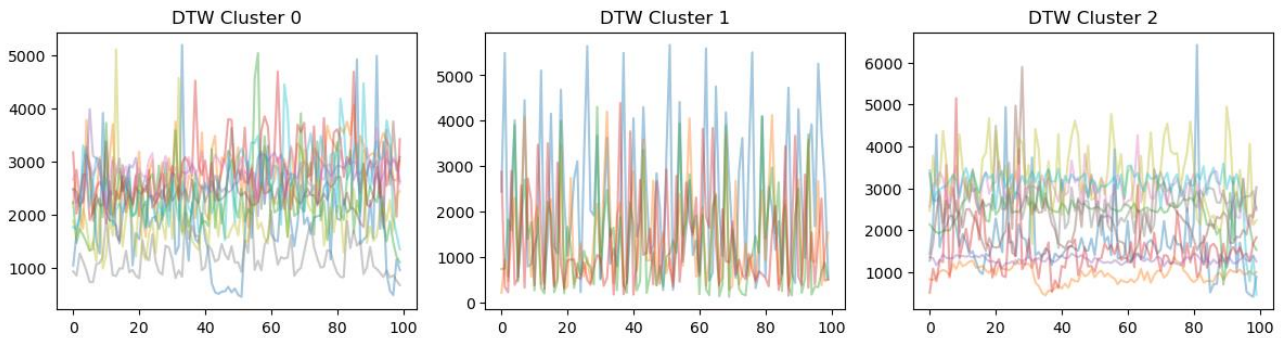


Table 12: Partitional Clustering with DTW Distance

To improve clustering efficiency, the dataset was initially reduced to a fraction of 10% of its original size, then increased to 70%. Using the tsfresh library, relevant features were extracted from the reduced time series, transforming them into a more manageable feature set. Columns with null values were removed to further clean the dataset. Subsequently, to tackle the challenge of handling excessively large datasets, approximation techniques such as SAX (Symbolic Aggregate approXimation) and PAA (Piecewise Aggregate Approximation) were utilized. These techniques allowed for dimensionality reduction while preserving essential features, thereby facilitating the clustering process and motif analysis.

An important step was the use of the Isolation Forest algorithm to identify and remove outliers from the dataset. Various contamination levels were tested to determine the best result in terms of Silhouette Score and Davies-Bouldin Index, resulting in the removal of 350 outliers. After removing the outliers, clustering was re-executed on the cleaned data using KMeans with both distance metrics, and the evaluation metrics were recalculated.

The analysis of descriptive statistics for each cluster provided a detailed overview of the characteristics of each group. For Euclidean distance, Cluster 0 showed some homogeneity with high maximum and minimum values, while Cluster 1 indicated high variability with generally lower maximum values. Cluster 2 demonstrated good homogeneity with high maximum values. For DTW distance, Cluster 0 showed good homogeneity, while Cluster 1, with only 4 elements, presented challenges in comparison. Cluster 2 displayed good homogeneity with high maximum values.

Despite these results, the initial evaluation metrics without preprocessing proved to be better than the values obtained after preprocessing. The Silhouette Score and the Davies-Bouldin Index for both Euclidean and DTW distances showed a significant deterioration after preprocessing. This led to the conclusion that preprocessing (dataset reduction, outlier removal, and PCA application) worsened the quality of the clusters. Therefore, it was decided to retain the original clusters without applying these preprocessing steps.

8.2.2 Hierarchical clustering

We describe here the process of analyzing and classifying time series using hierarchical clustering techniques. We began the analysis process by performing normalization to make the different time series comparable and to assign them the same weight during the clustering process. To facilitate the analysis and reduce the size of our dataset while retaining the main characteristics, we decided to use the Piecewise Aggregate Approximation (PAA) technique. Subsequently, several experiments were conducted using agglomerative clustering with different linkage metrics (ward, average, single).

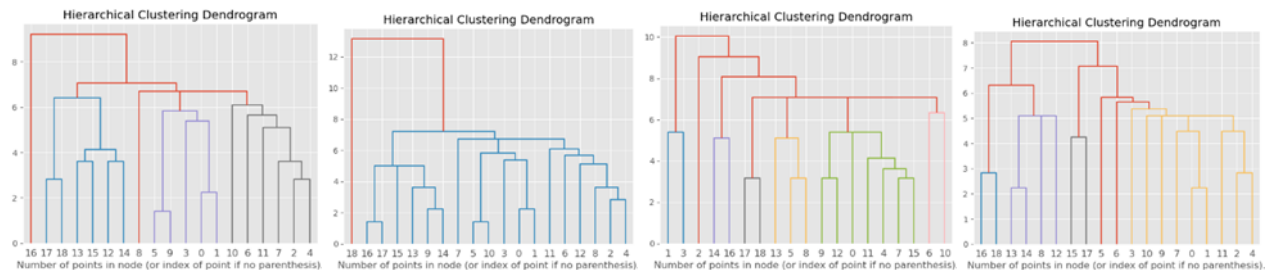


Table 14: Set of Hierarchical Clustering Dendrograms

The Ward linkage method was the first approach used for agglomerative clustering. This method minimizes the variance within clusters, creating relatively uniform cluster sizes. The resulting clusters were quite balanced, with a good distribution of time series among the clusters. However, the silhouette score obtained was 0.242, indicating that there was moderate internal cohesion of the clusters.

The second method considered was average linkage. This method calculates the average distance between all points of two clusters, thereby creating more rounded clusters compared to Ward linkage. Although the cluster distribution was still fairly balanced, the silhouette score obtained was lower than that of the Ward method, suggesting reduced internal cohesion.

The third method considered was single linkage, which merges clusters based on the minimum distance between any two points from different clusters. This method produced more compact clusters compared to the first two methods, with greater proximity among points. The silhouette score obtained was 0.35, the highest among the methods tested, indicating good internal cohesion of the clusters.

The last method tested was complete linkage, also known as the farthest linkage method, as it merges clusters based on the maximum distance between any two points from different clusters. This method tends to produce more compact and distinct clusters with well-defined boundaries between clusters. The silhouette score obtained was 0.32, indicating a good balance between cluster compactness and separation.

Hierarchical clustering has proven to be an effective tool for exploring the structure of time series data. However, the silhouette scores obtained indicate that none of the tested methods provided optimal results, suggesting that hierarchical clustering may not be the most effective method for our time series dataset.

8.2.3 Features-based clustering

For feature-based clustering, a systematic approach was undertaken involving the extraction of significant statistical features from complex time series. This process aimed to represent the time series in a more manageable and informative format. These are the features extracted:

- avg (media)
- std (deviazione standard)
- var (varianza)
- med (mediana)
- 10p (10° percentile)
- 25p (25° percentile)
- 50p (50° percentile, mediana)
- 75p (75° percentile)
- 90p (90° percentile)
- iqr (interquartile range)
- cov (coefficiente di variazione)
- skw (skewness)
- kur (kurtosis)

Once the statistical features were extracted from the time series, a thorough analysis was conducted to determine the optimal number of clusters (k). This was done using the elbow method, which evaluates the variation of the sum of squares within clusters as k varies. Additionally, various cluster validation criteria were considered and evaluated, including the Silhouette Score, Davies-Bouldin Index, and Dunn Index. These criteria provide a comprehensive assessment of cluster cohesion, separation, and distortion.

After careful analysis and comparing the results obtained with different values of k, the value 13 was selected as the optimal number of clusters. This decision was made considering a balance between the internal cohesion of clusters, their separation, and the interpretability of the results. The goal was to achieve meaningful and interpretable clusters that align with the characteristics of the analyzed time series data.



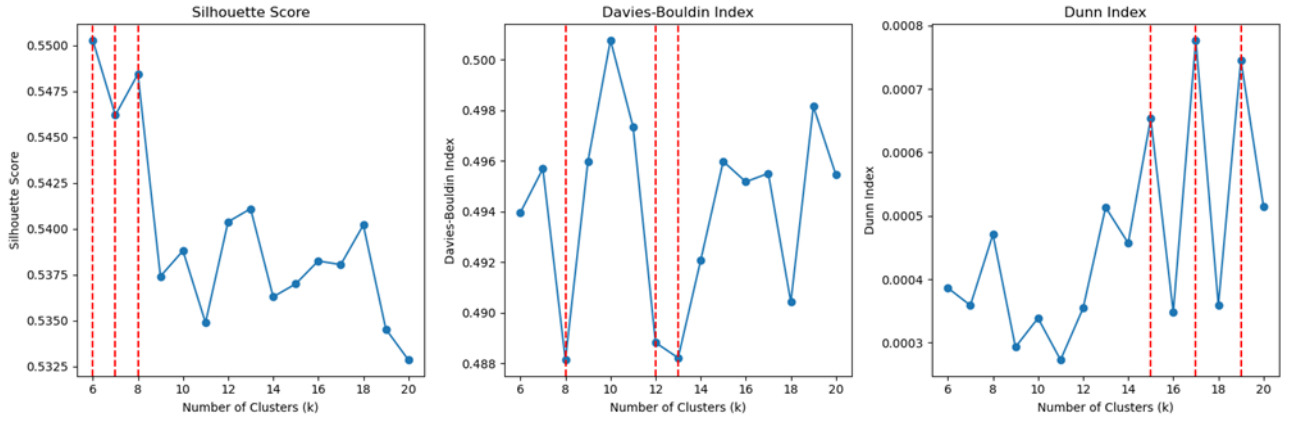


Table 16: Clustering Analysis

Unfortunately, the analysis of the obtained clusters revealed a significant disparity in the number of elements among the various groups. This phenomenon is not unique to the specific chosen value of k but seems to be intrinsic to the nature of the data and the extracted features. Upon closely examining the data distribution among clusters, it became apparent that some groups are considerably smaller and more isolated compared to others. This suggests the presence of subgroups or less frequent and distinct patterns within the analyzed dataset.

Furthermore, examining the distributions of musical genres within the clusters revealed significant heterogeneity. The genres do not exhibit specific or clear groupings within the identified clusters, indicating considerable variation in musical preferences represented across the different groups.

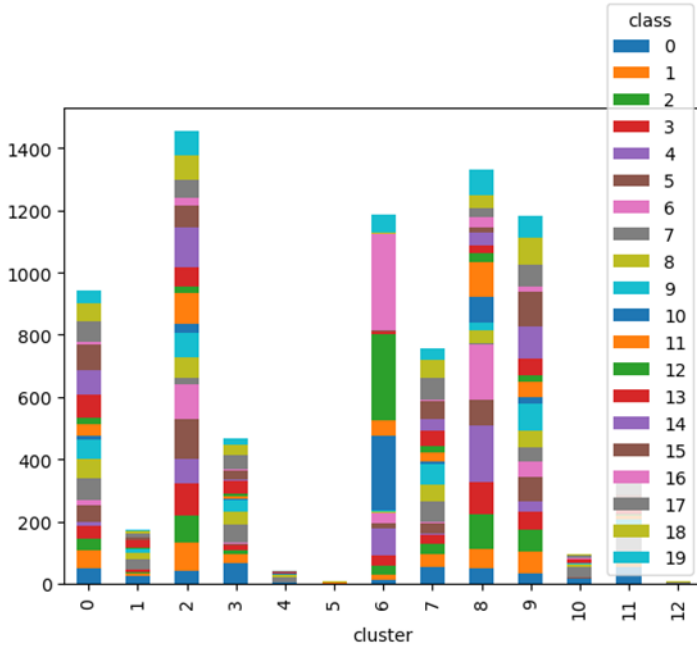


Figure 19: Features-based - clustering obtained

Silhouette Score (Test): 0.541093926985516

Davies-Bouldin Index (Test): 0.488217280099079

Adjusted Rand Index (Test): 0.053202692659707354

The Silhouette Score and Davies-Bouldin Index suggest that the clustering structure found is reasonably good in terms of cluster separation and compactness.

However, the Adjusted Rand Index indicates that there is little similarity between the predicted clustering labels and the true class labels, suggesting that the clusters identified by KMeans do not align well with the pre-existing categories in the data.

8.3 Classification

The goal of this study was to evaluate the performance of various classifiers on a synthetic dataset of time series. We experimented with different distance metrics and classifiers, including k-Nearest Neighbors (k-NN) with Euclidean, Manhattan, and Dynamic Time Warping (DTW) distances. Additionally, we explored more advanced techniques such as Random Forest and the ROCKET (RandOm Convolutional Kernel Transform) method.

8.3.1 Pre-processing

Since we observed that preprocessing worsened the quality of the clusters, we will use the original data without reducing the dataset, removing outliers, or applying PCA. The data will be normalized using MinMaxScaler to ensure all time series have values between 0 and 1.

8.3.2 Shapelets

Shapelets are subsequences of time series data that are highly discriminative for recognizing different classes. In this study, we used shapelets to perform the classification task on the target variable 'genre'. The goal is to assess the effectiveness of shapelets in improving the performance of classification models compared to a baseline classifier.

Our starting dataset consists of time series of varying lengths, each associated with a musical genre. Subsequently, we used SAX (Symbolic Aggregate approXimation) approximation with parameters $n_bins=5$ and $strategy='uniform'$ to transform the time series into a symbolic representation. This reduction in dimensionality of the time series facilitates analysis while preserving the main patterns and trends.

To establish a baseline, a Dummy Classifier was trained using the "most_frequent" strategy, which assigns the most frequent class in the training set to all instances. However, this classifier showed very low accuracy and precision, indicating that it is not effective at distinguishing between different genre classes.

The shapelets were extracted using the ShapeletModel from tslearn. Shapelets of length 50 were extracted, aligning with the length of the identified motifs. The model parameters included an SGD (Stochastic Gradient Descent) optimizer, a weight regularizer of 0.1, and a maximum number of iterations set to 200. These settings aimed to identify highly discriminative subsequences within the time series data that can help distinguish between different genre classes.

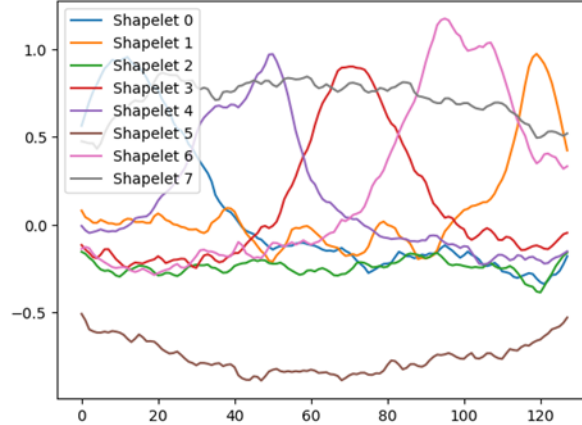


Table 17: Shapelets Analysis

The figure displays the different shapelets identified for each musical genre.

These shapelets were then used to transform the time series into a new representation based on distances from the shapelets themselves. This transformation enabled the training of more effective classification models compared to using the original data, although they were still not optimal. The performance of the shapelets was evaluated using various classification metrics including precision, recall, F1-score, and accuracy. In particular, the models showed improved performance compared to the baseline DummyClassifier, which achieved an accuracy of only 0.153 and a precision of 0.138.

Two classification models, KNN and Decision Tree, were trained using the shapelet transformation. The model parameters were optimized using RandomizedSearchCV. The KNN classifier was trained with the following values: $n_neighbors=50$, $weights='distance'$, $p=2$, $metric='euclidean'$. Despite achieving an accuracy of 0.16,

which is still low, there is already an improvement compared to the baseline model. For the Decision Tree Classifier, the following values were used: `max_depth=6`, `criterion='entropy'`, achieving an accuracy of 0.18.

The performance of the models was evaluated using precision, accuracy, and ROC curves. The performance analysis also included evaluation of the confusion matrices for both models, providing a detailed view of correctly and incorrectly classified classes.

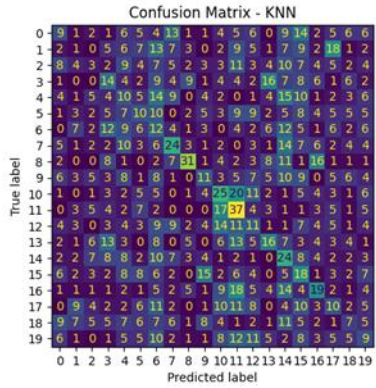


Table 18: Confusion Matrix - KNN

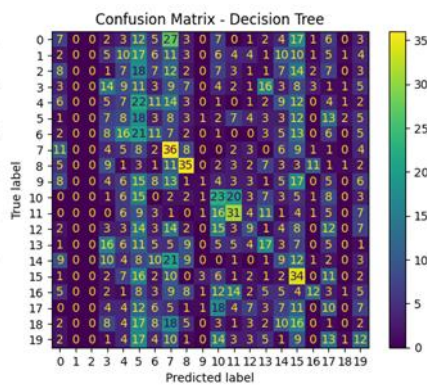


Table 19: Confusion Matrix – Decision Tree

Both confusion matrices highlight common issues in the KNN and Decision Tree classification models. Despite the improvement in performance compared to the baseline DummyClassifier due to shapelet transformation, the results show that both models struggle to correctly distinguish between the different classes, although the Decision Tree model appears to be more satisfactory.

For further evaluation of the models' performance in terms of class discrimination, ROC curves were used to assess the respective areas under the curve (AUC).

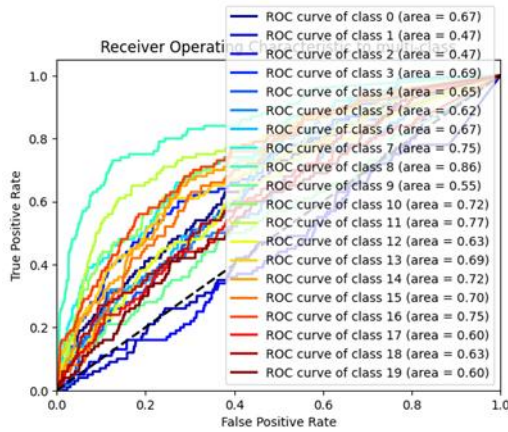


Figure 16: ROC – KNN

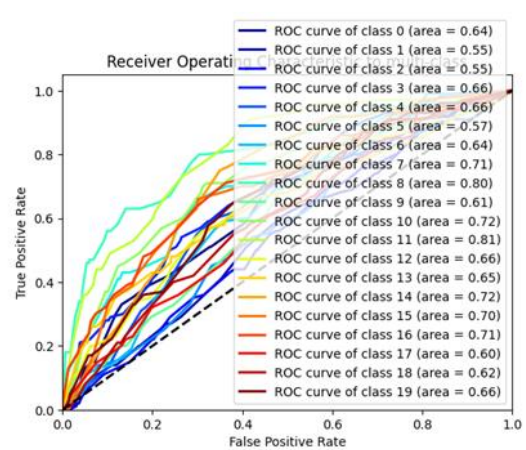


Figure 17: ROC- Decision Tree

The ROC curves for the KNN and Decision Tree models reveal that both models benefit from the shapelet transformation, significantly improving their classification capabilities compared to using the original data. However, both models show variable performance across different classes, with some classes being better discriminated than others.

KNN showed a high AUC for specific classes such as Blues and British, but struggled with others like Afrobeat and Alt-Rock. On the other hand, Decision Tree exhibited slightly better performance with a more consistent

AUC across classes. However, this model also faced challenges in discriminating certain classes such as Afrobeat and Alt-Rock.

These results suggest that classifying time series based on musical genres can be a complex problem. Differences between genres may not be distinct enough, and patterns in time series data may be too similar across different classes.

8.3.3 Comparison between Shapelets and Motifs

To evaluate the similarity between the extracted shapelets and the identified motifs of the time series, Euclidean distance was calculated. This comparison is crucial for understanding how representative the shapelets are compared to the known motifs of the time series.

Euclidean distance was used as the similarity metric. The distance between each extracted shapelet and the corresponding motifs was calculated to indicate their similarity. The results show the following Euclidean distances between the shapelets and motifs:

- Shapelet 1 e Motif 1: 3.48
- Shapelet 2 e Motif 2: 8.03
- Shapelet 3 e Motif 3: 5.88
- Shapelet 4 e Motif 4: 5.06
- Shapelet 5 e Motif 5: 3.22
- Shapelet 6 e Motif 6: 5.30

These distances might be influenced by the simplicity of Euclidean distance, which may not be optimal for capturing the relative similarities between the indices.

8.3.4 K-NN

Distance Metrics Used:

- Euclidean Distance: The k-NN classifier using Euclidean distance achieved an accuracy of 9.2%. This distance metric measures the square root of the sum of the squared differences between the coordinates of points in space. While it is one of the most commonly used metrics and intuitively simple, its relatively low performance in this context suggests that it does not adequately capture the temporal characteristics of the dataset.
- Manhattan Distance: It showed slightly better performance than Euclidean distance, with an accuracy of 9.3%. Manhattan distance, also known as block distance or taxi-cab distance, calculates the sum of the absolute differences of the coordinates. This marginally better result compared to Euclidean distance may indicate that in some cases, absolute linear differences are more informative than squared differences. However, overall, there was no substantial improvement in performance.
- Dynamic Time Warping (DTW): DTW significantly improved the performance of k-NN, achieving an accuracy of 21.2%. DTW is a technique for measuring the similarity between two time series that may vary in speed. It calculates the optimal alignment between the time series, allowing for variations in temporal scale. This result highlights the importance of considering temporal dynamics and nonlinear time variations for time series classification. The higher accuracy obtained with DTW suggests that it is more effective in capturing the intrinsic features of time series compared to traditional distance metrics.

The significant difference in accuracy between DTW and the other distance metrics underscores the importance of choosing the right distance metric for time series data. Whereas Euclidean and Manhattan distances fail to account for temporal variations, DTW successfully aligns time series, enhancing the model's ability to distinguish between different classes.

This analysis suggests that for time series classification problems, distance metrics that account for nonlinear temporal variations, such as DTW, can offer significant advantages.

8.3.5 Random Forest

Random Forest is an ensemble method that combines predictions from multiple decision trees. This approach mitigates the impact of outliers and data variability, enhancing the overall robustness of the model. Decision trees are capable of capturing nonlinear relationships between variables. Random Forest achieved an accuracy of 25.45%, surpassing the k-NN methods. However, precision, recall, and F1-score varied across classes, indicating some challenges in distinguishing certain classes.

Variability in Precision, Recall, and F1-score Between Classes

- **Imbalanced Class Distribution:** If the dataset exhibits an imbalanced distribution among classes, the model may struggle to correctly identify the less represented classes. This can lead to significant variations in precision, recall, and F1-score metrics, where classes with fewer instances are often less accurately predicted by the model due to less training data.
- **Intrinsic Challenges of Some Classes:** Some classes may possess highly similar characteristics, making it challenging for the model to distinguish between them accurately. This similarity can result in lower precision, recall, and F1-score for those classes as the model may struggle to identify subtle differences between them.

In summary, both imbalanced class distributions and intrinsic similarities between classes can contribute to variability in performance metrics such as precision, recall, and F1-score in classification tasks. Addressing these challenges often involves careful dataset balancing strategies and feature engineering to enhance the model's ability to distinguish between classes effectively.

8.3.6 Gradient Boosting

Gradient Boosting achieved an accuracy of 24.4%, slightly lower than Random Forest, which achieved 25.45%. This method combines predictions from numerous weak decision trees to form a robust model, optimizing residual errors iteratively. However, the sequential nature of Gradient Boosting can make it more susceptible to overfitting compared to Random Forest, which constructs trees in parallel and averages their predictions to mitigate overfitting.

Similar to Random Forest, Gradient Boosting exhibited variations in precision and recall metrics among different classes. The main reasons for these variations include:

- **Imbalanced Class Distribution:** When there is an unequal distribution of classes in the dataset, the model may struggle to accurately predict the less represented classes, leading to varying precision and recall metrics across classes.
- **Intrinsic Challenges of Some Classes:** Some classes may have similar characteristics, making it difficult for the model to distinguish between them accurately. This similarity can contribute to lower precision and recall scores for those classes.
- **Model Complexity:** The choice of parameters such as the number of trees, maximum depth, and learning rate significantly influences the model's performance. Suboptimal parameter configurations can reduce the model's effectiveness, resulting in lower performance compared to Random Forest.

In summary, while Gradient Boosting can yield competitive performance, it requires careful parameter tuning to balance model complexity and prevent overfitting, especially when faced with imbalanced class distributions and intrinsic class similarities.

8.3.7 ROCKET

The ROCKET method was initially implemented using RidgeClassifierCV. However, the initial results were disappointing, with an accuracy of 5%. This low accuracy was attributed to various factors, including potential issues in the data transformation process and the compatibility of the classifier used.

To improve performance, RidgeClassifierCV was replaced with RandomForestClassifier, which proved to be more suitable for leveraging the transformations performed by ROCKET. This transition was motivated by RandomForestClassifier's ability to handle large amounts of variables derived from kernel transformations, as well as its robustness against overfitting compared to RidgeClassifierCV.

8.3.8 Conclusioni

Classifiers k-NN: Although easy to implement, k-NN classifiers with Euclidean and Manhattan distances showed poor performance. DTW significantly improved performance, highlighting the importance of considering temporal dynamics in time series classification.

Random Forest and Gradient Boosting: Both ensemble methods outperformed k-NN, with Random Forest exhibiting the best overall performance. This underscores the robustness of ensemble methods in handling complex datasets.

ROCKET Method: Despite its potential, the initial application of ROCKET did not yield satisfactory results. This could be attributed to issues with transformation or the choice of post-transformation classifier. Further adjustments and experimentation with different approaches are needed.

In conclusion, Random Forest emerged as the best classifier for this dataset, achieving an accuracy of 25.45% and surpassing k-NN and other advanced techniques.

8.4 Motifs and Discords

In this analysis, various clustering techniques were employed to analyze time series data using both Euclidean and DTW (Dynamic Time Warping) distances. Evaluation metrics were computed without any initial preprocessing, yielding the following results: for Euclidean distance, the Silhouette Score was 0.2224 and the Davies-Bouldin Index was 1.4420, while for DTW distance, the Silhouette Score was 0.2194 and the Davies-Bouldin Index was 1.4418.

Subsequently, motifs within each cluster were analyzed. In Cluster 0, 219,088 motifs were identified before filtering, and the top 5 motifs with the highest correlation were selected. In Cluster 1, 84,346 motifs were found before filtering, and similarly, the top 5 motifs by correlation were chosen. Finally, in Cluster 2, 208,795 motifs were identified before filtering, and again, the top 5 motifs by correlation were selected.

Here are some examples of the selected motifs analyzed in detail:

In time series 1284, motifs were identified between indices 3-13 and 35-45 with a correlation of 0.999673, between indices 5-15 and 37-47 with a correlation of 0.999547, and between indices 4-14 and 36-46 with a correlation of 0.999452.

In time series 2616, motifs were found between indices 18-28 and 43-53 with a correlation of 0.999489, and between indices 23-33 and 48-58 with a correlation of 0.999452.

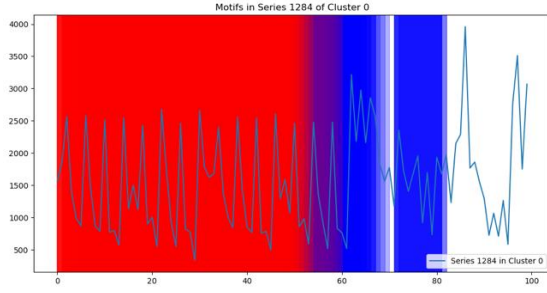


Figure 18: Motifs in Series 1284 of Cluster 0

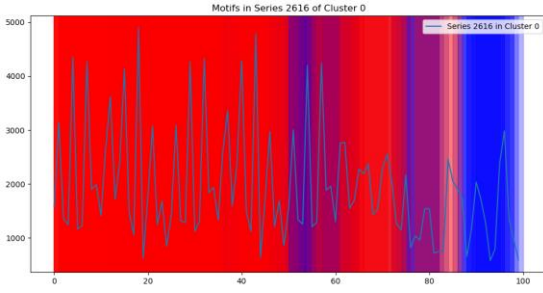


Figure 19: Motifs in Series 2616 of Cluster 0

The displayed images show the clusters obtained using both distances. These visualizations are useful for better understanding how the data has been divided into different clusters. Overall, the analysis revealed that the selected motifs are highly correlated, indicating strong similarities among segments of the time series within the clusters.

In conclusion, the results show that even without preprocessing, the formed clusters are of high quality, as indicated by the high correlations among the selected motifs. This motif analysis provides valuable insights into repetitive patterns present in the time series data, which can be used for further advanced analyses and applications.

Additionally, motif analysis was applied to the previously identified clusters. The function used loaded and preprocessed the time series data, resizing them to ensure they all had the same length. Motifs were identified using the matrix profile method, and the top motifs were selected based on their highest correlation. The analysis revealed highly similar motifs, indicating significant repetitive patterns in the time series. Some time series, such as series 189 and series 1933, were particularly rich in repetitive patterns.

In summary, the analysis has led to the discovery of significant clusters in the time series data, after removing outliers and optimizing clustering parameters. However, it was found that preprocessing worsened the quality of the clusters, suggesting that the original clusters should be retained for future analyses. The identification of motifs has further contributed to pinpointing meaningful repetitive patterns, offering new opportunities for advanced analyses.

9. Explainability

Model interpretability is crucial for understanding and trusting the decisions made by machine learning models. In this analysis, we chose to use XGBoost, a powerful boosting algorithm based on trees, well-regarded for its efficiency, flexibility, and generalization capabilities. To train the model, we set the maximum depth of each tree (`max_depth`) to 5, and the learning rate (`learning_rate`) was fixed at 0.1, allowing the model to update weights slowly and precisely at each boosting round.

Regarding the number of trees in the model (`n_estimators`), we chose to use 200 trees. Additionally, we decided to use only 80% of the samples to train each tree (`subsample`). Early stopping was set to 10, meaning the training would stop if the performance on the validation set did not improve for 10 consecutive rounds, thereby preventing overfitting.

9.1 Global Methods

Global methods provide an overview of the model's behavior, explaining how the model uses features to make predictions across the entire dataset.

Feature Importance

First, we considered the importance of features by assigning scores based on their ability to improve the accuracy of the model's predictions and understand which of these features have the greatest impact on predictions.

From the results, we observed that the most influential features were "acousticness," "speechiness," "instrumentalness," and "album_total_tracks." These variables play a crucial role in the model's predictions, suggesting that aspects such as acoustic quality, presence of speech, instrumental content, and the total number of tracks in an album are significant determinants in our context.

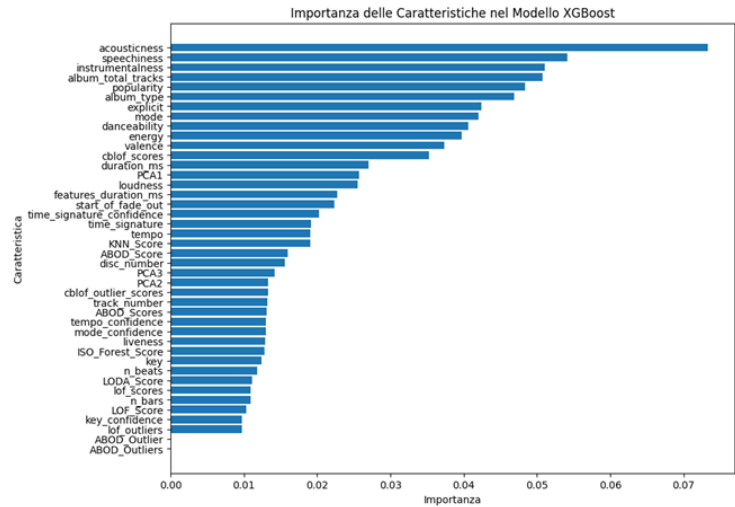


Figure 20: Feature Importance with XGBoost

9.1.1 Trepan (Tree-based Rule Extraction)

Trepan is an algorithm that extracts a decision tree from a complex model, approximating the behavior of the original model. This decision tree simplifies the interpretation of how the model makes decisions.

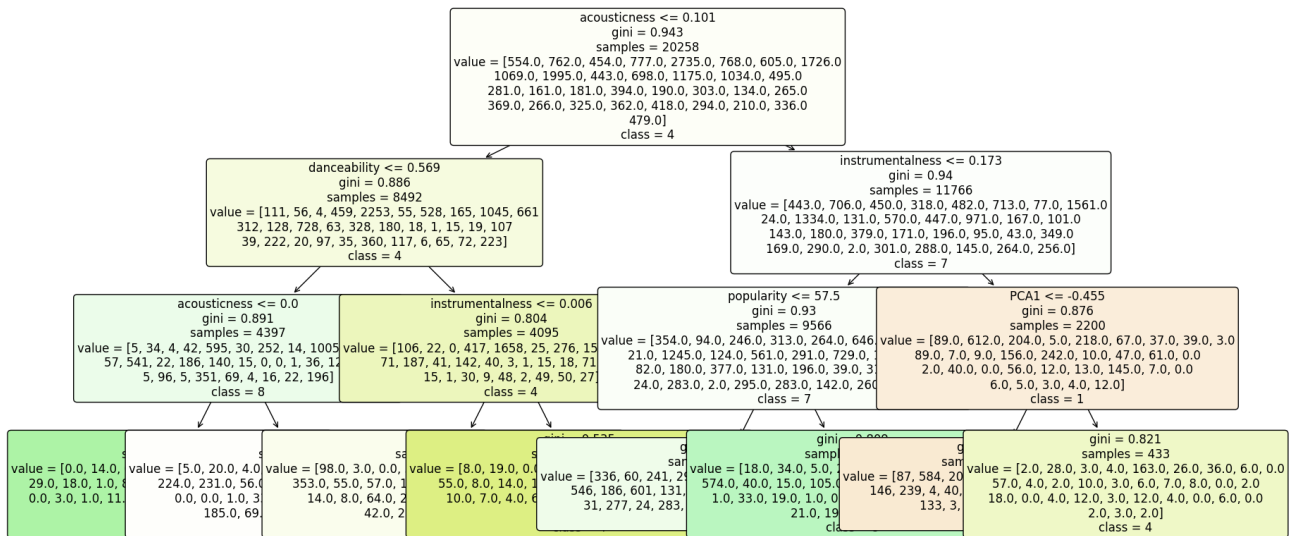


Figure 21: Decision Tree with Trepan

The decision tree shows how various features influence the model predictions. For example, we can observe that "acousticness" and "danceability" are among the key features determining the initial classification. At deeper levels of the tree, other features such as "instrumentalness" and "popularity" become relevant for further decisions.

9.2 Local Methods

Local Methods focus on explanations of individual predictions, helping us understand why the model made a specific prediction for a particular example.

9.2.1 SHAP Values (SHapley Additive exPlanations)

The SHAP values provide a consistent and unified explanation of how each feature contributes to the prediction of a single data point and across the entire dataset.

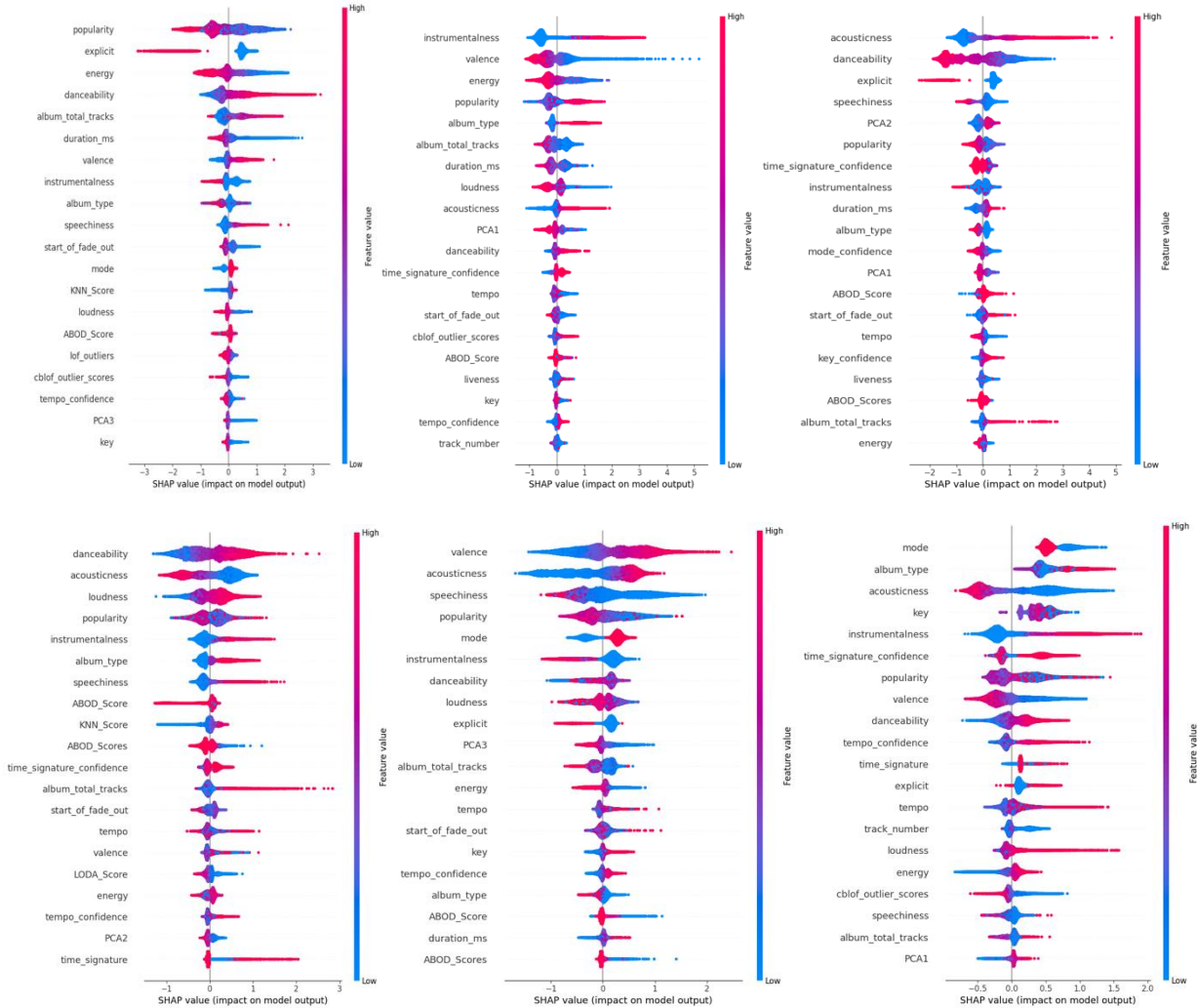


Figure 19: SHAP-impact of features

From the SHAP plots, we can see those features such as "popularity," "explicit," "energy," and "danceability" have a significant impact. This approach has allowed us to visualize not only the importance of these features but also the direction of their effect (positive or negative) on predictions. For example, in the SHAP plots, we observe that high values of "popularity" and "danceability" tend to have a positive impact on predictions, whereas features like "explicit" and "ABOD_Score" show variable impacts depending on the specific context.

9.2.2 LIME (Local Interpretable Model-agnostic Explanations)

LIME is a technique that explains model predictions by training an interpretable local model (such as a linear model or tree) around a specific prediction. We applied LIME to analyze predictions made by our XGBoost model on individual examples. Below is an example of a LIME plot for a specific prediction from the XGBoost model: in this example, we see how different features influence the model's prediction probabilities. "instrumentalness" and "valence" are among the most influential features, with "instrumentalness" contributing the most to the prediction. Other factors like "duration_ms" and "energy" have a more limited impact.

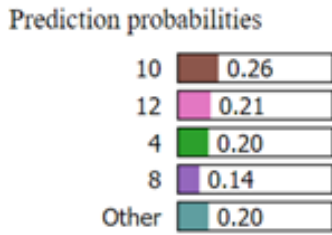


Figure 20: LIME- Predicted class membership probability

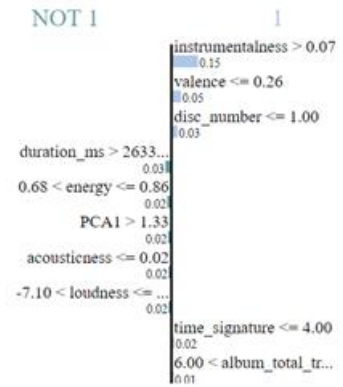


Figure 21: LIME- Features that influence classification

Feature	Value
instrumentalness	0.33
valence	0.15
disc_number	1.00
duration_ms	276226.00
energy	0.69
PCA1	1.34
acousticness	0.00
loudness	-7.03
time_signature	4.00

Figure 22: Actual feature values that influence the prediction

9.3 Conclusion

From the global and local analysis, we have gained a comprehensive understanding of the model's behavior. Global methods showed us the most influential features across the entire dataset, while local methods allowed us to understand the model's decisions at the level of individual examples. The combined use of these techniques revealed that certain features, such as "acousticness" and "speechiness," consistently play an important role, while other features may have a variable impact depending on the specific context of each example.

To evaluate the model's performance, we used various classification metrics. The classification report indicated that the XGBoost model achieved excellent results, with an accuracy of 92%. Other metrics such as precision, recall, and F1-score further confirmed the model's robustness, with values of 0.90, 0.91, and 0.91 respectively. These results indicate that the model is highly effective in correctly predicting classes, demonstrating a good balance between precision and recall.